SONIC-MCU

RV32IM RISC-V System-on-Chip

Data Sheet | DS-SONICMCU-v1.2

About This Document

This document provides the architectural specification, interface descriptions, memory map, boot flow, and peripheral reference for the **SONIC-MCU** RV32IM SoC, developed by the **RISCForge** team at the **Indian Institute of Information Technology, Allahabad (IIITA)**.

Document Number:	DS-SONICMCU-v1.2
Revision:	1.2 — RISC-V Core Update
Date:	March 2026
ISA:	RISC-V RV32IM
Core Clock:	100 MHz
Bus Fabric:	AXI-Lite Crossbar

Team — RISCForge, IIITA

Members

1. [Member Name 1]
2. [Member Name 2]
3. [Member Name 3]
4. [Member Name 4]
5. [Member Name 5]
6. [Member Name 6]
7. [Member Name 7]
8. [Member Name 8]

Faculty Advisors

1. [Faculty Advisor 1]
2. [Faculty Advisor 2]

Mentor

1. [Mentor Name]

Copyright © 2026 RISCForge, IIITA. All rights reserved. All specifications subject to change without notice.

Contents

1 Introduction 3

2 Features 3

3 RISC-V Core Specification 4
3.1 Top-Level Ports . . . 4
3.2 Pipeline Stage Overview . . . 4
3.3 Pipeline Architecture . . . 5
3.4 Fetch Stage — `fetch_cycle` . . . 5
3.5 Decode Stage — `decode_cycle` . . . 6
3.6 Execute Stage — `execute_cycle` . . . 6
3.7 Memory Stage — `memory_cycle` . . . 7
3.8 Write-Back Stage — `writeback_cycle` . . . 8

4 Control Unit 8
4.1 Control_Unit_Top . . . 8
4.2 Main Decoder Truth Table . . . 9
4.3 ALU Decoder Encoding . . . 10

5 ALU and Hazard Unit 10
5.1 ALU . . . 10
5.2 Hazard Unit — `hazard_unit` . . . 10

6 Register File and Memory Subsystem 11
6.1 Register File — `Register_File` . . . 11
6.2 Load/Store Unit — `ls_unit` . . . 12
6.3 Immediate Generator — `Sign_Extend` . . . 13

7 Supported Instruction Set 13
7.1 R-Type Instructions (opcode `0110011`) . . . 14
7.2 I-Type Instructions (opcode `0010011`) . . . 14
7.3 Load Instructions (opcode `0000011`) . . . 15
7.4 Store Instructions (opcode `0100011`) . . . 15
7.5 Branch Instructions (opcode `1100011`) . . . 15
7.6 Jump and Upper-Immediate Instructions . . . 15
7.7 RV32M — Multiply/Divide Extension . . . 15

8 Functional Description 16
8.1 AXI-Lite Bus Masters . . . 16
8.2 Memory Subsystem . . . 16
8.3 AXI-Lite Crossbar (Fabric) . . . 16
8.4 APB Peripheral Subsystem . . . 16
8.5 Boot Controller . . . 16

9 Address Map 17

10 Clocks and Resets 17

11 Interrupts 18
11.1 CLIC-to-Core Interface Signals . . . 18

12 Boot Flow 18
12.1 Overview . . . 18
12.2 Boot Controller State Machine . . . 18
12.3 Boot Software Sequence . . . 19
12.4 SPI-AXI Initiator Registers (Base: `0x4000_6000`) . . . 19

13 Core Interface Signals **20**
13.1 Instruction Port (I-BUS) . . . 20
13.2 Data Port (D-BUS / LSU) . . . 20
13.3 AXI-Lite Boot DMA Adapter Signals . . . 20

14 Peripheral IP Cores **21**
14.1 GPIO . . . 21
14.2 UART . . . 21
14.3 SPI Slave . . . 21
14.4 I2C . . . 22
14.5 Timer . . . 22
14.6 CLIC . . . 22

15 Known Limitations (v1.2) **22**

16 Revision History **23**

Introduction

SONIC-MCU is a 32-bit RISC-V SoC implementing the RV32IM ISA, developed by the RISCForge team at IIITA. The SoC features a 5-stage in-order pipeline running at 100 MHz, a 32 KB instruction SRAM, 32 KB data SRAM, and a 4 KB mask-programmed Boot ROM. It targets general-purpose bare-metal embedded applications including sensing, control, and communication.

The SoC interconnect is an **AXI-Lite Crossbar** fabric that routes transactions between two AXI masters (the processor core and the boot-time DMA initiator), memories, and an APB peripheral subsystem via an AXI-Lite-to-APB bridge. All peripheral IP cores are sourced from the open-source PULP platform libraries.

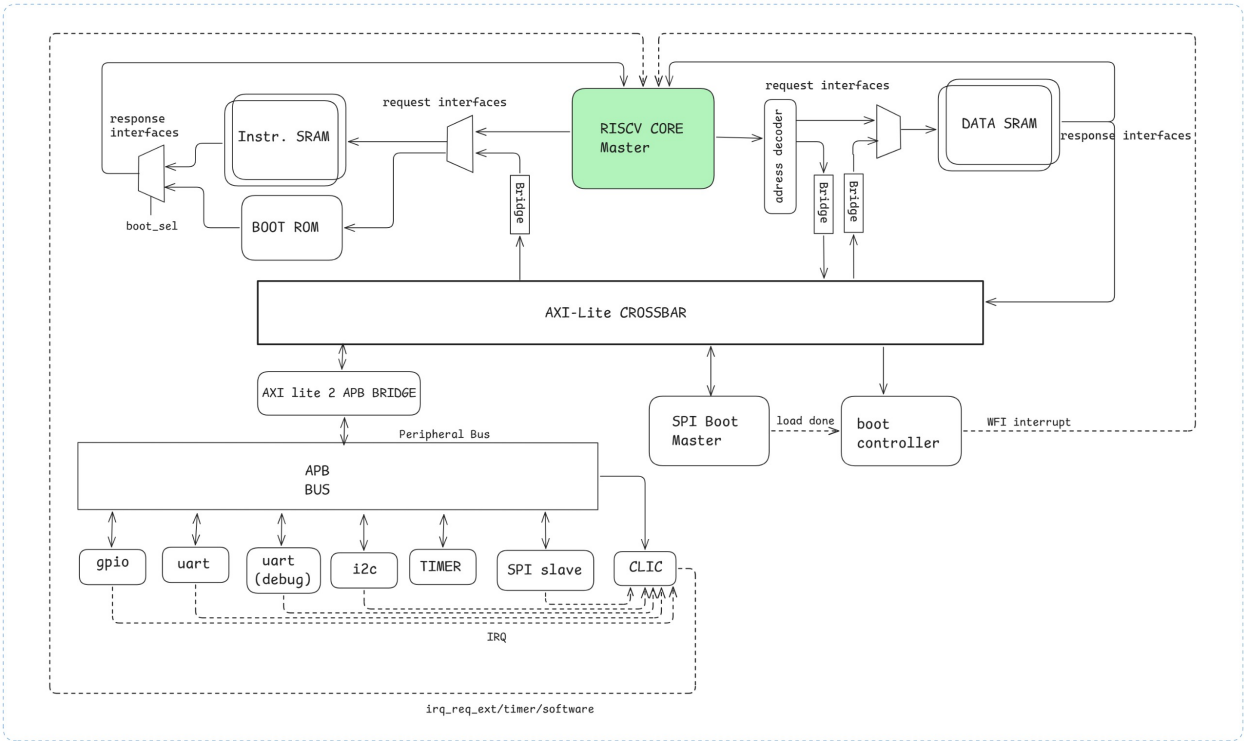


Figure 1: SONIC-MCU Top-Level Architecture

Features

- **RISC-V RV32IM** core — integer and multiply/divide extensions, machine-mode privilege
- **5-stage in-order pipeline:** IF, ID, EX, MEM, WB
- Full data-forwarding network: MEM-to-EX and WB-to-EX bypass paths
- Load-use hazard detection with automatic 1-cycle pipeline stall
- Control hazard handling: 2-stage pipeline flush on taken branch or jump
- Illegal opcode detection with Execute-stage flush
- **100 MHz** system clock frequency
- 32 $\times$ 32-bit register file; two async read ports, one sync write port; x0 hardwired to zero
- **Memory subsystem:**
  - 32 KB Instruction SRAM (I-SRAM) at `0x0000_9000`
  - 32 KB Data SRAM (D-SRAM) at `0x0000_1000`

  - 4 KB Boot ROM at `0x0000_0000`
- **Dual AXI-Lite masters** on the crossbar fabric:
  - Master 0: RISC-V core (normal program execution)
  - Master 1: SPI-AXI Initiator (boot-time DMA image loading)
- **AXI-Lite Crossbar** interconnect with round-robin arbitration and address decoding
- **AXI-Lite to APB bridge** for peripheral subsystem access
- **Hardware boot controller** with SPI-AXI initiator for autonomous firmware loading
- **Peripherals (PULP open-source IPs):**
  - 16-pin GPIO
  - UART $\times 2$ (application + debug)
  - SPI Slave
  - I2C Master
  - 32-bit Timer
  - CLIC (Core-Local Interrupt Controller)
- RISC-V `WFI` instruction support

RISC-V Core Specification

Top-Level Ports

The top-level module `top` instantiates all five pipeline stage modules and the hazard unit. All pipeline registers use an active-low asynchronous reset.

Table 1: Top-Level Port List (`module top`)

Signal	Direction	Width	Description
`clk`	Input	1	System clock
`rst`	Input	1	Active-low asynchronous global reset
`PCSrcE1`	Output	1	Debug: branch/jump taken (mirrors PCSrcE from Execute stage)

Pipeline Stage Overview

Table 2: Pipeline Stage Summary

Stage	Module	Key Responsibilities
IF — Fetch	`fetch_cycle`	Maintain PC, select next PC (sequential vs. target), fetch instruction from I-MEM, produce PC+4
ID — Decode	`decode_cycle`	Decode instruction, read register file, generate immediates, produce all EX-stage control signals
EX — Execute	`execute_cycle`	Apply forwarding, execute ALU/jump/branch, compute branch target, produce EX/MEM pipeline register outputs
MEM — Memory	`memory_cycle`	Perform data memory reads/writes, apply `ls_unit` formatting, latch results into MEM/WB registers
WB — Write-Back	`writeback_cycle`	Select final result (ALU, memory read, or PC+4) and return it to the register file

Pipeline Architecture

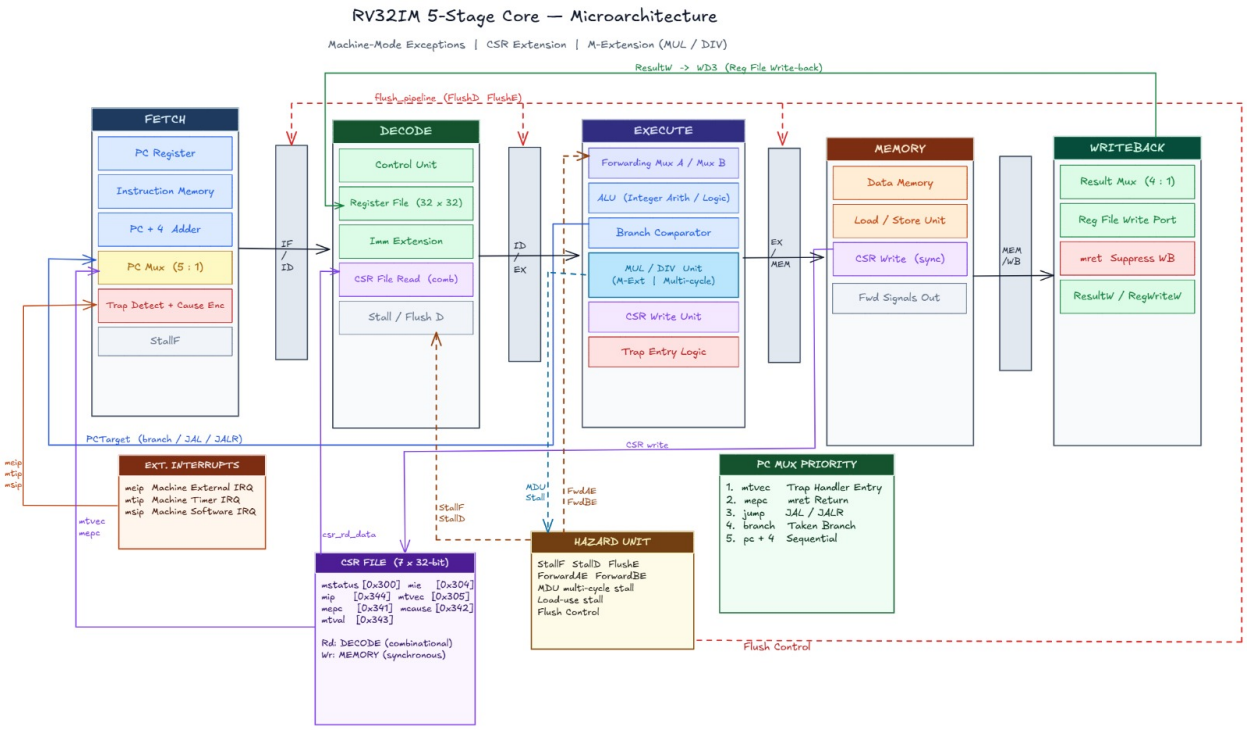


Figure 2: SONIC-MCU RV32IM 5-Stage Pipeline Architecture

Fetch Stage — `fetch_cycle`

The Fetch stage maintains the program counter and issues an instruction fetch every cycle. The PC is updated at the rising clock edge unless a stall is asserted. On `FlushD` (taken branch/jump), the IF/ID register is zeroed, inserting a NOP bubble.

Table 3: `fetch_cycle` Port List

Signal	Direction	Width	Description
`clk`	Input	1	System clock
`rst`	Input	1	Active-low async reset — clears IF/ID register
`PCSrcE`	Input	1	0 = sequential (PC+4), 1 = branch/jump target
`PCTargetE`	Input	32	Branch/jump target address from Execute stage
`StallF`	Input	1	Freeze PC register (load-use stall)
`StallD`	Input	1	Freeze IF/ID pipeline register
`FlushD`	Input	1	Clear IF/ID register (control hazard)
`InstrD`	Output	32	Instruction word forwarded to Decode stage
`PCD`	Output	32	Current PC forwarded to Decode stage
`PCPlus4D`	Output	32	PC+4 forwarded to Decode stage

Decode Stage — `decode_cycle`

The Decode stage reads two source registers, generates the sign/zero-extended immediate, and produces all downstream control signals via the Control Unit. `FlushE` zeroes all ID/EX register fields, inserting a bubble into the Execute stage.

Table 4: `decode_cycle` Port List

Signal	Direction	Width	Description
`clk` / `rst`	Input	1	Clock / active-low async reset
`InstrD`	Input	32	Instruction word from IF stage
`PCD` / `PCPlus4D`	Input	32	PC and PC+4 from IF stage
`RegWriteW`	Input	1	WB-stage register write enable
`RDW`	Input	5	WB-stage destination register address
`ResultW`	Input	32	WB-stage write-back value
`FlushE`	Input	1	Flush ID/EX register
`RegWriteE`	Output	1	Register write enable propagated to EX
`ALUSrcE`	Output	1	ALU B source: 0=register, 1=immediate
`MemWriteE`	Output	1	Data memory write enable to EX
`ResultSrcE`	Output	2	WB mux: 00=ALU, 01=MEM read, 10=PC+4
`JumpE`	Output	2	[1]=jump enable, [0]=JAL(1)/JALR(0)
`BranchE`	Output	4	[3]=branch enable, [2:0]=funct3 type
`ALUControlE`	Output	4	ALU operation select
`lsunitE`	Output	5	Load/Store control bus to EX
`RD1_E` / `RD2_E`	Output	32	Register file read data (rs1, rs2)
`Imm_Ext_E`	Output	32	Sign-extended immediate
`RD_E`	Output	5	Destination register address (rd)
`RS1_E` / `RS2_E`	Output	5	Source register addresses to hazard unit
`RS1_D` / `RS2_D`	Output	5	Source register addresses in Decode stage
`PCE` / `PCPlus4E`	Output	32	PC and PC+4 propagated to EX
`LUISrcE`	Output	1	Op[5]: distinguishes LUI from AUIPC
`isLUIE`	Output	1	U-type indicator: 1 for LUI/AUIPC
`illegal_instr`	Output	1	Illegal opcode flag from `opcode_check`

Execute Stage — `execute_cycle`

The Execute stage applies forwarding muxes to both ALU operands, executes the ALU operation, computes branch/jump targets, evaluates branch conditions via `JB_unit`, and handles the LUI/AUIPC result override. The forwarded register value is used as store data to avoid stale-value bugs.

Table 5: `execute_cycle` Port List

Signal	Direction	Width	Description
`clk` / `rst`	Input	1	Clock / active-low async reset
`RegWriteE`	Input	1	Register write enable
`ALUSrcE`	Input	1	ALU B-src select: 0=register, 1=immediate
`MemWriteE`	Input	1	Memory write enable
`LUISrcE`	Input	1	LUI src: 1=use Imm, 0=PC+Imm (AUIPC)
`isLUIE`	Input	1	Override ALU result with LUI/AUIPC result
`ResultSrcE`	Input	2	WB result mux select
`BranchE`	Input	4	Branch control bus
`ALUControlE`	Input	4	ALU operation select
`JumpE`	Input	2	Jump control bus
`lsunitE`	Input	5	Load/Store unit control bus
`RD1_E` / `RD2_E`	Input	32	Register read data from Decode stage
`Imm_Ext_E`	Input	32	Sign-extended immediate
`RD_E`	Input	5	Destination register (rd) address
`PCE` / `PCPlus4E`	Input	32	Current PC and PC+4
`ResultW`	Input	32	WB-stage result (forwarding path 01)
`ForwardA_E`	Input	2	Forwarding select for ALU src A
`ForwardB_E`	Input	2	Forwarding select for ALU src B
`PCSrcE`	Output	1	1 = branch or jump taken
`PCTargetE`	Output	32	Next PC if branch/jump taken
`RegWriteM`	Output	1	Propagated to MEM stage
`MemWriteM`	Output	1	Propagated to MEM stage
`ResultSrcM`	Output	2	Propagated to MEM stage
`RD_M`	Output	5	Destination register to MEM stage
`lsunitM`	Output	5	Load/Store control bus to MEM stage
`PCPlus4M`	Output	32	PC+4 propagated to MEM stage
`WriteDataM`	Output	32	Forwarded store data to MEM stage
`ALU_ResultM`	Output	32	ALU/LUI/AUIPC result to MEM stage

Memory Stage — `memory_cycle`

The Memory stage performs all data memory accesses. For stores it passes aligned data and byte-enable strobes from `ls_unit` to `Data_Memory`. For loads it formats the raw 32-bit memory word into the correct byte/halfword/word size with sign or zero extension.

Table 6: `memory_cycle` Port List

Signal	Direction	Width	Description
`clk` / `rst`	Input	1	Clock / active-low async reset
`RegWriteM`	Input	1	Register write enable from EX
`MemWriteM`	Input	1	Memory write enable
`ResultSrcM`	Input	2	WB result mux select
`RD_M`	Input	5	Destination register address
`lsunitM`	Input	5	Load/Store control bus
`PCPlus4M`	Input	32	PC+4 from EX stage
`WriteDataM`	Input	32	Store data (forwarded in EX stage)
`ALU_ResultM`	Input	32	Effective memory address from ALU
`RegWriteW`	Output	1	Propagated to WB stage
`ResultSrcW`	Output	2	Propagated to WB stage
`RD_W`	Output	5	Destination register to WB stage
`PCPlus4W`	Output	32	PC+4 propagated to WB stage
`ALU_ResultW`	Output	32	ALU result propagated to WB stage
`ReadDataW`	Output	32	Formatted memory read data to WB stage

Write-Back Stage — `writeback_cycle`

The Write-Back stage is purely combinational. It selects the result to write back to the register file from three sources based on `ResultSrcW`, then drives `ResultW` back into the Decode stage.

Table 7: `writeback_cycle` Port List

Signal	Direction	Width	Description
`ResultSrcW`	Input	2	Result source: 00=ALU, 01=MEM read, 10=PC+4
`ALU_ResultW`	Input	32	ALU result from MEM/WB register
`ReadDataW`	Input	32	Formatted memory read data
`PCPlus4W`	Input	32	PC+4 for JAL/JALR link address
`ResultW`	Output	32	Write-back value; returned to register file and forwarding path

Control Unit

Control_Unit_Top

The top-level control unit is purely combinational. It instantiates two sub-decoders — `Main_Decoder` and `ALU_Decoder` — and assembles three composite control buses: `Branch[3:0]`, `Jump[1:0]`, and `lsunit[4:0]`.

Table 8: `Control_Unit_Top` Port List

Signal	Direction	Width	Description
`Op`	Input	7	Instruction opcode field [6:0]
`funct3`	Input	3	Instruction bits [14:12]
`funct7_30`	Input	1	Instruction bit [30] (SUB/SRA detection)
`RegWrite`	Output	1	Register file write enable
`ImmSrc`	Output	2	Immediate format: 00=I, 01=S, 10=B, 11=J
`ALUSrc`	Output	1	ALU B source: 0=register, 1=immediate
`MemWrite`	Output	1	Data memory write enable
`ResultSrc`	Output	2	WB mux: 00=ALU, 01=MEM read, 10=PC+4
`Branch`	Output	4	[3]=enable, [2:0]=funct3 branch type
`Jump`	Output	2	[1]=jump enable, [0]=JAL(1)/JALR(0)
`lsunit`	Output	5	[4]=LS enable, [3]=Op[5], [2:0]=funct3
`ALUControl`	Output	4	Precise ALU operation
`LUISrcE`	Output	1	Op[5]: distinguishes LUI from AUIPC
`isLUIE`	Output	1	1 when instruction is LUI or AUIPC
`illegal_instr`	Output	1	1 when opcode does not match any valid RV32I opcode

Main Decoder Truth Table

Table 9: Main Decoder — Opcode to Control Signals

Opcode	Type	RegWr	ALUSrc	MemWr	ResultSrc	ImmSrc	ALUOp	Br/Jmp
0110011	R-type	1	0	0	00	00	10	—
0010011	I-ALU	1	1	0	00	00	10	—
0000011	LOAD	1	1	0	01	00	00	—
0100011	STORE	0	1	1	00	01	00	—
1100011	BRANCH	0	0	0	00	10	01	B=1
1101111	JAL	1	1	0	10	11	00	J=1
1100111	JALR	1	1	0	10	00	11	J=1
0110111	LUI	1	1	0	00	00	–	isLUI=1
0010111	AUIPC	1	1	0	00	00	–	isLUI=1

ALU Decoder Encoding

Table 10: ALU Control Encoding

ALUControl	Operation	ALUOp	funct3	funct7[30]/op[5]
0000	ADD	00/10	000	{0,x} or {1,0}
0001	SUB	01/10	000/001	{1,1} (R-SUB)
0100	AND	10	111	x
0101	OR	10	110	x
0110	JALR	11	—	—
0111	XOR	10	100	x
1000	SLL	10	001	x
1001	SRL	10	101	0
1010	SRA	10	101	1
1100	SLTU	01/10	011/110-111	x
1111	SLT	01/10	010/100-101	x

ALU and Hazard Unit

ALU

The ALU is a purely combinational 32-bit datapath unit. The Zero flag is asserted when Result == 0 and is used by `JB_unit` to resolve BEQ/BNE/BGE/BGEU conditions.

Note

For JALR (`ALUControl = 0110`), the ALU computes `(A + B) & ~1`, which clears the LSB per the RISC-V specification.

Table 11: ALU Port List

Signal	Direction	Width	Description
`A`	Input	32	First operand (forwarded register value Src_A)
`B`	Input	32	Second operand (Src_B: register or immediate)
`ALUControl`	Input	4	Operation select
`Result`	Output	32	Computed result
`Zero`	Output	1	1 when Result == 32'h0

Hazard Unit — `hazard_unit`

The hazard unit is purely combinational and monitors pipeline register contents to manage all three hazard classes: data hazards (forwarding), load-use hazards (stalling), and control hazards (flushing).

Table 12: `hazard_unit` Port List

Signal	Direction	Width	Description
`rst`	Input	1	Active-low reset — forces all forward selects to 00
`RegWriteM`	Input	1	Register write enable of MEM-stage instruction
`RegWriteW`	Input	1	Register write enable of WB-stage instruction
`RD_M`	Input	5	Destination register of MEM-stage instruction
`RD_W`	Input	5	Destination register of WB-stage instruction
`Rs1_E` / `Rs2_E`	Input	5	Source registers of EX-stage instruction
`Rs1_D` / `Rs2_D`	Input	5	Source registers of Decode-stage instruction
`RdE`	Input	5	Destination register of EX-stage instruction
`ResultSrcE`	Input	2	EX-stage result source: 2'b01 flags a load
`PCSrcE`	Input	1	Taken branch/jump from Execute stage
`illegal_instr`	Input	1	Illegal opcode detected in Decode stage
`ForwardAE`	Output	2	Forward mux select for ALU operand A
`ForwardBE`	Output	2	Forward mux select for ALU operand B
`StallF`	Output	1	Stall PC and IF/ID register
`StallD`	Output	1	Stall IF/ID register
`FlushE`	Output	1	Flush ID/EX register
`FlushD`	Output	1	Flush IF/ID register (taken branch/jump)

Table 13: Forwarding Mux Encoding

ForwardAE/Forwar(	Source	Condition
`2'b00`	Register file (RD1_E/RD2_E)	No hazard; use decoded register value
`2'b01`	ResultW (WB stage)	RegWriteW=1, RD_W == Rs_E, RD_W $\neq$ x0
`2'b10`	ALU_ResultM (MEM stage)	RegWriteM=1, RD_M == Rs_E, RD_M $\neq$ x0 (higher priority)

Register File and Memory Subsystem

Register File — `Register_File`

32×32-bit. Two fully asynchronous read ports, one synchronous write port. x0 is hardwired to zero. Same-cycle write forwarding on both read ports.

Table 14: `Register_File` Port List

Signal	Direction	Width	Description
`clk`	Input	1	Write clock (rising edge)
`rst`	Input	1	Active-low reset — read ports output 0
`WE3`	Input	1	Write enable
`A1` / `A2`	Input	5	Read port addresses (rs1, rs2)
`A3`	Input	5	Write port address (rd)
`WD3`	Input	32	Write data
`RD1`	Output	32	Read data for rs1 (async, with write forwarding)
`RD2`	Output	32	Read data for rs2 (async, with write forwarding)

Load/Store Unit — `ls_unit`

Handles sub-word memory access alignment: load data formatting (sign/zero extension), store byte-enable generation, and store data alignment.

Table 15: `ls_unit` Port List

Signal	Direction	Width	Description
`in`	Input	32	Raw data word read from Data_Memory
`address`	Input	2	Effective address bits [1:0] for alignment
`in_s`	Input	32	Store register value (forwarded Src_B_interim)
`lsunit`	Input	5	[4]=LS enable, [3]=Op[5], [2:0]=funct3
`out`	Output	32	Formatted load result
`strobe`	Output	4	Byte-enable signals for Data_Memory WSTRB
`out_s`	Output	32	Aligned store data for Data_Memory WD

Table 16: Load/Store Unit Operation Encoding

lsunit[4:3]	lsunit[2:0]	Operation	Notes
10	000	LB — Load Byte	Sign-extend to 32 bits
10	001	LH — Load Halfword	Sign-extend to 32 bits
10	010	LW — Load Word	Full 32-bit pass-through
10	100	LBU — Load Byte Unsigned	Zero-extend to 32 bits
10	101	LHU — Load Halfword Unsigned	Zero-extend to 32 bits
11	000	SB — Store Byte	Byte lane selected by address[1:0]
11	001	SH — Store Halfword	Halfword lane selected by address[1]
11	010	SW — Store Word	All four byte lanes enabled

Immediate Generator — `Sign_Extend`

Table 17: Immediate Format Encoding

ImmSrc/LUI	Format	Bit Extraction	Description
11	J-type	{In[31]×12, In[19:12], In[20], In[30:21], 1'b0}	JAL 21-bit signed offset
10	B-type	{In[31]×20, In[7], In[30:25], In[11:8], 1'b0}	Branch 13-bit signed offset
01	S-type	{In[31]×20, In[31:25], In[11:7]}	Store 12-bit signed offset
LUI=1	U-type	{In[31:12], 12'b0}	Upper 20 bits, lower zeroed
Shift I	Shift	In[24:20]	SLLI/SRLI/SRAI shift amount
00	I-type	{In[31]×20, In[31:20]}	Default 12-bit signed immediate

Supported Instruction Set

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

Figure 3: RV32I Instruction Formats

RV32M Multiply Extension

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
mul	MUL	R	0110011	0x0	0x01	rd = (rs1 * rs2)[31:0]
mulh	MUL High	R	0110011	0x1	0x01	rd = (rs1 * rs2)[63:32]
mulsu	MUL High (S) (U)	R	0110011	0x2	0x01	rd = (rs1 * rs2)[63:32]
mulu	MUL High (U)	R	0110011	0x3	0x01	rd = (rs1 * rs2)[63:32]
div	DIV	R	0110011	0x4	0x01	rd = rs1 / rs2
divu	DIV (U)	R	0110011	0x5	0x01	rd = rs1 / rs2
rem	Remainder	R	0110011	0x6	0x01	rd = rs1 % rs2
remu	Remainder (U)	R	0110011	0x7	0x01	rd = rs1 % rs2

Figure 4: RV32M Instruction Format

R-Type Instructions (opcode 0110011)

Table 18: R-Type Instructions

Mnemonic	funct3	funct7[30]	Operation
ADD	000	0	rd = rs1 + rs2
SUB	000	1	rd = rs1 - rs2
AND	111	0	rd = rs1 & rs2
OR	110	0	rd = rs1 | rs2
XOR	100	0	rd = rs1 ^ rs2
SLL	001	0	rd = rs1 << rs2[4:0]
SRL	101	0	rd = rs1 >> rs2[4:0] (logical)
SRA	101	1	rd = rs1 >>> rs2[4:0] (arithmetic)
SLT	010	0	rd = (rs1 < rs2) ? 1 : 0 (signed)
SLTU	011	0	rd = (rs1 < rs2) ? 1 : 0 (unsigned)

I-Type Instructions (opcode 0010011)

Table 19: I-Type ALU Immediate Instructions

Mnemonic	funct3	Operation
ADDI	000	rd = rs1 + imm
ANDI	111	rd = rs1 & imm
ORI	110	rd = rs1 | imm
XORI	100	rd = rs1 ^ imm
SLLI	001	rd = rs1 << imm[4:0]
SRLI	101	rd = rs1 >> imm[4:0] (logical, imm[10]=0)
SRAI	101	rd = rs1 >>> imm[4:0] (arithmetic, imm[10]=1)
SLTI	010	rd = (rs1 < imm) ? 1 : 0 (signed)
SLTIU	011	rd = (rs1 < imm) ? 1 : 0 (unsigned)

Load Instructions (opcode 0000011)

Table 20: Load Instructions

Mnemonic	funct3	Operation
LB	000	rd = SignExt(MEM[rs1+imm][7:0])
LH	001	rd = SignExt(MEM[rs1+imm][15:0])
LW	010	rd = MEM[rs1+imm][31:0]
LBU	100	rd = ZeroExt(MEM[rs1+imm][7:0])
LHU	101	rd = ZeroExt(MEM[rs1+imm][15:0])

Store Instructions (opcode 0100011)

Table 21: Store Instructions

Mnemonic	funct3	Operation
SB	000	MEM[rs1+imm][7:0] = rs2[7:0]
SH	001	MEM[rs1+imm][15:0] = rs2[15:0]
SW	010	MEM[rs1+imm][31:0] = rs2[31:0]

Branch Instructions (opcode 1100011)

Table 22: Branch Instructions

Mnemonic	funct3	Condition
BEQ	000	PC += imm if rs1 == rs2
BNE	001	PC += imm if rs1 $\neq$ rs2
BLT	100	PC += imm if rs1 $<$ rs2 (signed)
BGE	101	PC += imm if rs1 $\geq$ rs2 (signed)
BLTU	110	PC += imm if rs1 $<$ rs2 (unsigned)
BGEU	111	PC += imm if rs1 $\geq$ rs2 (unsigned)

Jump and Upper-Immediate Instructions

Table 23: Jump and Upper-Immediate Instructions

Mnemonic	Opcode	Operation
JAL	1101111	rd = PC+4; PC = PC + imm (J-type offset)
JALR	1100111	rd = PC+4; PC = (rs1 + imm) & ~1
LUI	0110111	rd = imm $<<$ 12 (upper 20 bits, lower zero)
AUIPC	0010111	rd = PC + (imm $<<$ 12)

RV32M — Multiply/Divide Extension

Note

RV32M multiply/divide is **planned** for a future revision. All M-extension instructions share opcode 0110011 with R-type and are distinguished by `funct7 = 7'b0000001`. The ALU_Decoder will be extended to recognise this pattern and route to a dedicated multiply/divide unit.

Functional Description

AXI-Lite Bus Masters

The AXI-Lite Crossbar operates with **two masters** at any given time:

Table 24: AXI-Lite Masters

Master	Source	Active During
0	RISC-V Core	Normal program execution; all load/store and peripheral access
1	SPI-AXI Initiator (Boot DMA)	Boot phase only; writes firmware from SPI flash into I-SRAM and D-SRAM; core stalled (STALL=1) during this phase

During boot, Master 1 has exclusive use of the crossbar while the CPU pipeline is frozen. Once the Boot Controller transitions to DONE, Master 1 becomes inactive and all subsequent transactions originate from Master 0.

Memory Subsystem

The instruction-side memory controller arbitrates between two sources using `instr_ram_mux`:

- **Port 0 (AXI adapter):** AXI-Lite writes from the boot DMA initiator during firmware loading
- **Port 1 (Core fetch):** Instruction fetches from the pipeline IF stage

Address bit `[ADDR_WIDTH-1]` selects between Boot ROM (`is_boot=1`) and I-SRAM (`is_boot=0`). Port 0 has higher priority; core fetches stall for the duration of any AXI write beat.

The data-side controller uses an **address DEMUX** to route LSU transactions: D-SRAM range addresses go directly to the SRAM adapter; all others are forwarded to the AXI-Lite crossbar.

AXI-Lite Crossbar (Fabric)

The interconnect is the PULP `axi_lite_xbar`, implementing address decoder, request demux, response mux, and round-robin arbiter.

Note

AXI-Lite only — no bursts, IDs, or out-of-order transactions. All bus widths are 32-bit data.

APB Peripheral Subsystem

Peripherals connect via **APB** through the `axi_lite_to_apb` bridge, translating AXI-Lite transactions into `PSEL`, `PENABLE`, `PWRITE`, `PADDR`, `PWDATA` sequences.

Boot Controller

A dedicated hardware boot controller sequences SoC startup. It controls `STALL`, `BOOT_SEL`, and `DATA_MUX_SEL`. See Section 12 for full details.

Address Map

Table 25: SONIC-MCU Default Address Map

Device	Base Address	Size	Region	Notes
Boot ROM	0x0000_0000	4 KB	Cache	Default fetch on reset
D-SRAM	0x0000_1000	32 KB	Cache	Data memory
I-SRAM	0x0000_9000	32 KB	Cache	Program entry point
GPIO	0x1A00_0000	4 KB	I/O	16-pin
UART (App)	0x1A10_0000	4 KB	I/O	
UART (Debug)	0x1A11_0000	4 KB	I/O	
I2C	0x1A20_0000	4 KB	I/O	
Timer	0x1A30_0000	4 KB	I/O	
SPI Slave	0x1A40_0000	4 KB	I/O	
CLIC	0x1A50_0000	64 KB	I/O	
SPI Boot Ctrl	0x4000_6000	4 KB	I/O	SPI-AXI Initiator

Note

Always use `volatile` casts when accessing I/O region addresses:

```
*((volatile uint32_t *) 0x1A10_0000);   // UART base example
```

Clocks and Resets

Table 26: Clock Ports

Port	Direction	Width	Frequency	Description
clk_i	Input	1	100 MHz	System clock: core, crossbar, memories
pclk_i	Input	1	50 MHz	Peripheral clock for APB bus and IPs
rst_ni	Input	1	—	Active-low asynchronous global reset

Table 27: Reset Ports

Port	Direction	Width	Description
rst_ni	Input	1	Active-low async reset for entire SoC
core_rst_n	Output	1	Synchronous reset to RISC-V core; held low until Boot Controller reaches IDLE
per_rst_n	Output	1	Synchronous reset for all APB peripherals

Interrupts

Table 28: CLIC Interrupt Source Table

IRQ ID	Source	Trigger	Description
0	UART (App)	Level	RX ready / TX empty
1	UART (Debug)	Level	Debug receive ready
2	SPI Slave	Level	Transfer complete
3	I2C	Level	Transfer complete / NACK
4	GPIO	Edge	Configurable edge per pin
5	Timer	Level	Timer limit reached
6	Software	—	CLIC software interrupt

CLIC-to-Core Interface Signals

Table 29: CLIC-to-Core Interrupt Signals

Signal	Direction	Width	Description
`irq_valid_o`	Output	1	Interrupt pending
`irq_id_o[4:0]`	Output	5	Interrupt source ID
`irq_level_o[7:0]`	Output	8	Interrupt priority level
`irq_shv_o`	Output	1	Selective hardware vectoring flag
`irq_priv_o[1:0]`	Output	2	Privilege mode of interrupt
`irq_ready_i`	Input	1	Core ready to accept interrupt
`irq_kill_req_o`	Output	1	Core preempted by higher-priority IRQ
`irq_kill_ack_i`	Input	1	Core confirms preemption

Boot Flow

Overview

On power-on reset the core fetches from Boot ROM at `0x0000_0000`. The Boot ROM initialises the SPI peripheral and programs the SPI-AXI Initiator (Master 1), which autonomously transfers the firmware from SPI flash into I-SRAM and D-SRAM. The core is stalled during this transfer. After completion the Boot Controller de-asserts `STALL`, sets `BOOT_SEL`=1, and the core jumps to `0x0000_9000`.

Boot Controller State Machine

Table 30: Boot Controller States

State	BOOT_SEL	STALL	DATA_MUX_SEL	Description
IDLE	0	0	0	Reset state; core fetching Boot ROM
TRANSFER	0	1	1	DMA in progress; CPU pipeline frozen
DONE	1	0	0	Transfer complete; core running I-SRAM

Table 31: Boot Controller State Transitions

From	To	Condition	Action
IDLE	TRANSFER	awvalid=1 AND awaddr=SPI_DMA_START	Assert STALL; assert DATA_MUX_SEL
TRANSFER	DONE	DONE=1 from SPI-AXI Initiator	Deassert STALL; set BOOT_SEL=1
DONE	DONE	Always	Latch permanently

Boot Software Sequence

1. **Configure SPI** via APB (0x1A40_0000) — write clock divider, mode, enable
2. **Program SPI-AXI Initiator** — write destination 0x0000_9000 and transfer length
3. **Trigger DMA** — write 1 to SPI_DMA_START; execute WFI to drain pipeline before STALL asserts
4. **Resume** — core resumes after WFI when STALL de-asserts
5. **Setup vectors** — write mtvec; enable MIE
6. **Jump** to 0x0000_9000

Listing 1: Boot ROM Assembly Sequence

```
li   t0, SPI_BASE
li   t1, CLK_DIV_VAL
sw   t1, SPI_CLK_DIV_REG(t0)
sw   t1, SPI_MODE_REG(t0)
sw   t1, SPI_ENABLE_REG(t0)

li   t0, SPI_DMA_BASE          # 0x40006000
li   t1, 0x00009000
sw   t1, SPI_DMA_DST_ADDR(t0)
li   t1, 0x00010000
sw   t1, SPI_DMA_LEN(t0)
li   t1, 0x1
sw   t1, SPI_DMA_START(t0)
wfi                            # STALL asserts here

li   t0, MTVEC_VAL
csrw mtvec, t0
li   t0, MSTATUS_MIE
csrs mstatus, t0
li   t0, 0x00009000
jr   t0
```

SPI-AXI Initiator Registers (Base: 0x4000_6000)

Table 32: SPI-AXI Initiator Register Map

Offset	Register	Width	Access	Reset	Description
0x00	SPI_DMA_DST_ADDR	32	RW	0x0	Destination base address
0x04	SPI_DMA_LEN	32	RW	0x0	Transfer length in bytes
0x08	SPI_DMA_START	1	RW	0x0	Write 1 to trigger DMA
0x0C	SPI_DMA_STATUS	2	RO	0x0	Bit[0]=DONE, Bit[1]=ERROR

Important

Boot Controller has **no timeout**. If SPI transfer stalls, `STALL` is held permanently until reset. Error detection reserved for a future revision.

Core Interface Signals

Instruction Port (I-BUS)

Table 33: Core Instruction Interface (I-BUS)

Signal	Direction	Width	Description
`core_instr_req`	Output	1	Instruction fetch request
`core_instr_addr[31:0]`	Output	32	Fetch address
`core_instr_gnt`	Input	1	Grant: memory accepted request
`core_instr_rvalid`	Input	1	Return data valid
`core_instr_rdata[31:0]`	Input	32	Fetched instruction word

Data Port (D-BUS / LSU)

Table 34: Core Data Interface (LSU / D-BUS)

Signal	Direction	Width	Description
`core_data_req`	Output	1	Load/store request
`core_data_addr[31:0]`	Output	32	Target address
`core_data_we`	Output	1	Write enable (1=write)
`core_data_wdata[31:0]`	Output	32	Write data
`core_data_be[3:0]`	Output	4	Byte enable strobes
`core_data_gnt`	Input	1	Grant from demux/crossbar
`core_data_rvalid`	Input	1	Read data valid
`core_data_rdata[31:0]`	Input	32	Read data from memory/peripheral

AXI-Lite Boot DMA Adapter Signals

Table 35: AXI Boot DMA Adapter to Instruction RAM

Signal	Direction	Width	Description
`axi_instr_req`	Output	1	AXI adapter memory request
`axi_instr_addr[31:0]`	Output	32	Target address
`axi_instr_we`	Output	1	Write enable
`axi_instr_be[3:0]`	Output	4	Byte enables
`axi_instr_wdata[31:0]`	Output	32	Write data (AXI W-channel)

Peripheral IP Cores

Bridge source: https://github.com/pulp-platform/axi/blob/master/src/axi_lite_to_apb.sv

Table 36: Peripheral IP Summary

Peripheral	Count	Usage	GitHub Source
GPIO	1	16-pin digital I/O	pulp-platform/gpio
UART	2	App + debug	pulp-platform/apb_uart
SPI Slave	1	Boot / comms	pulp-platform/apb_spi_master
I2C	1	Sensor I2C master	pulp-platform/apb_i2c
Timer	1	System tick	pulp-platform/apb_timer
CLIC	1	Interrupt controller	pulp-platform/clic

GPIO

Source: https://github.com/pulp-platform/gpio

16-pin bidirectional GPIO. Pin count is chosen to stay within the total I/O pin budget of the target device. All 16 pins are independently configurable as input or output. Edge-detection enables rising/falling-edge interrupt generation to CLIC IRQ 4.

Table 37: GPIO Signals (Base: 0x1A00_0000)

Signal	Direction	Width	Description
gpio_in[15:0]	Input	16	GPIO input pins
gpio_out[15:0]	Output	16	GPIO output pins
gpio_dir[15:0]	—	16	Direction register (1=output)
gpio_intr_o	Output	1	Edge interrupt to CLIC

UART

Source: https://github.com/pulp-platform/apb_uart

Two instances: UART 0 (0x1A10_0000) for application, UART 1 (0x1A11_0000) for debug.

Table 38: UART Signals

Signal	Direction	Width	Description
uart_rx_i	Input	1	UART receive line
uart_tx_o	Output	1	UART transmit line
uart_intr_o	Output	1	RX/TX interrupt to CLIC

SPI Slave

Source: https://github.com/pulp-platform/apb_spi_master

Table 39: SPI Slave Signals (Base: 0x1A40_0000)

Signal	Direction	Width	Description
spi_sck_i	Input	1	SPI clock from master
spi_cs_i	Input	1	Chip select (active low)
spi_mosi_i	Input	1	Master Out Slave In
spi_miso_o	Output	1	Master In Slave Out
spi_intr_o	Output	1	Transfer complete interrupt to CLIC

Table 40: SPI Slave Register Map

Offset	Register	Width	Access	Description
0x00	SPI_TXDATA	32	RW	Transmit data register
0x04	SPI_RXDATA	32	RO	Receive data register
0x08	SPI_STATUS	2	RO	Bit[0]=TX empty, Bit[1]=RX valid

I2C

Source: https://github.com/pulp-platform/apb_i2c
APB I2C master supporting 100 kHz and 400 kHz modes. Transfer-complete and NACK events generate IRQ 3 to CLIC.

Table 41: I2C Signals

Signal	Direction	Width	Description
i2c_scl_io	Inout	1	I2C clock line (open-drain)
i2c_sda_io	Inout	1	I2C data line (open-drain)
i2c_intr_o	Output	1	Transfer complete / NACK interrupt to CLIC

Timer

Source: https://github.com/pulp-platform/apb_timer
32-bit compare timer for system tick and general-purpose timing.

Table 42: Timer Signals

Signal	Direction	Width	Description
timer_intr_o	Output	1	Compare-match interrupt to CLIC

Table 43: Timer Register Map (Base: 0x1A30_0000)

Offset	Register	Width	Access	Description
0x00	TIMER_CTRL	32	RW	Start/stop, prescaler enable
0x04	TIMER_PRESCALE	32	RW	Clock divider value
0x08	TIMER_CMP	32	RW	Compare/limit value
0x0C	TIMER_VALUE	32	RO	Current counter value

CLIC

Source: https://github.com/pulp-platform/clic
Aggregates up to 16 interrupt sources with programmable priority levels. Priority arbitration is hardware-managed; the highest-priority pending interrupt is presented via irq_id_o and irq_level_o.

Known Limitations (v1.2)

1. Boot mode is **hardwired** to SPI. No runtime mode selection.

2. `SPI_DMA_STATUS` ERROR bit is **reserved**; transfer error detection not implemented.
3. Boot Controller has **no timeout** — STALL held permanently if SPI transfer stalls until reset.
4. **No secure boot**: binary loaded without authentication. Planned for a future revision.
5. **RV32M** multiply/divide extension is planned; not yet implemented in the ALU.

Revision History

Version	Date	Author	Changes
1.0	March 2026	RISCForge Team	Initial release
1.1	March 2026	RISCForge Team	Removed prefetch and FENCE; clarified dual AXI masters; GPIO pin count note
1.2	March 2026	RISCForge Team	RISC-V Core Update: full pipeline stage specs, control unit tables, hazard unit, ALU, register file, load/store unit, and complete RV32I instruction set added; Width column added to all signal tables